



23. 動態規劃

2026資訊研究社算法班
Made with ❤️ by jheanlee



動態規劃是什麼

動態規劃(dynamic programming) 的目標：

- 將一個複雜的問題切分成簡單的子問題
- 避免重複計算
- 優化子結構

簡單來說就是對程式問題賦予數學式的解法

動態規劃有兩種思考方式 :top-down 與 bottom-up



動態規劃實例—費氏數列 (top-down)

費氏數列的第 n 項為：第 $n-1$ 項 + 第 $n-2$ 項；而第 0 項定義為 0、第 1 項定義為 1

透過遞迴方式我們可以快速地寫出解法：

```
int basic_fib(int n) {  
    if (n <= 1) return n;  
    return basic_fib(n - 1) + basic_fib(n - 2);  
}
```

動態規劃實例—費氏數列 (top-down)

但如果我們將這個遞迴樹展開，我們會發現有許多步驟其實是重複計算的。

以 `basic_fib(5)` 為例 (`basic_fib(n)` 省略為 `f(n)`):

1. $f(5)$
2. $f(4) + f(3)$
3. $(f(3) + f(2)) + (f(2) + f(1))$
4. $(f(2) + f(1)) + (f(1) + f(0)) + (f(1) + f(0)) + f(1)$
5. $((f(1) + f(0)) + f(1)) + (f(1) + f(0)) + (f(1) + f(0)) + f(1)$



動態規劃實例—費氏數列 (top-down)

這時候我們可以使用一個表格幫我們記錄下已經處理過的資料。

這個技巧稱為 memoisation

```
int memo_fib(int n, vector<int> &memo) {  
    if (n <= 1) return n;  
    if (memo[n] != -1) return memo[n];  
    memo[n] = basic_fib(n - 1) + basic_fib(n - 2);  
    return memo[n];  
}
```

動態規劃實例—費氏數列 (bottom-up)

如果我們反過來思考：在多層遞迴後，我們建構出了一個 memo 表。那我們能不能直接將答案建構於表上而捨棄遞迴呢？

這便是 bottom-up 寫法的精神，我們透過一個陣列(可能多維)，將解答直接建構出來。

這個寫法是從第 0 項開始向上建構，與遞迴相反。

```
int bottom_up_fib(int n) {  
    vector<int> dp(n + 1);  
  
    // 初始條件  
    dp[0] = 0;  
    dp[1] = 1;  
  
    // 開始迭代  
    for (int i = 2; i <= n; i++) {  
        dp[i] = dp[i - 1] + dp[i - 2];  
    }  
  
    return dp[n];  
}
```



動態規劃實例—費氏數列 (bottom-up)

Bottom-up 寫法有兩個要點

1. 初始條件 (base case)
2. 一般項的定義

只要搞清楚這兩點便能輕鬆寫出動態規劃解法
(雖然要想到很難啦)

動態規劃實例—費氏數列 (bottom-up 降維)

有時我們直覺下所想到的解法並非最有效率的解法。必須在其中看出其他浪費掉的空間或時間並進行優化。

費氏數列計算時僅需要前兩項的資料即可，不需要所有項的資料。因此我們可以僅保留前兩項於變數中，將空間複雜度自 $O(n)$ 降為 $O(1)$ 。

```
int bottom_up_optimised_fib(int n) {  
    if (n <= 1) return n;  
  
    int prev = 0; int curr = 1;  
    for (int i = 2; i <= n; i++) {  
        int tmp = prev + curr;  
        prev = curr;  
        curr = tmp;  
    }  
  
    return curr;  
}
```




Leetcode 70 - Climbing Stairs

這題是經典的爬樓梯問題。有一個 n 階的樓梯，一步可以踩一階或跨兩階。在只往上走的狀況下，請問幾種到頂的方式？

Example 1:

Input: $n = 2$

Output: 2

Explanation: There are two ways to climb to the top.

1. 1 step + 1 step
2. 2 steps

Example 2:

Input: $n = 3$

Output: 3

Explanation: There are three ways to climb to the top.

1. 1 step + 1 step + 1 step
2. 1 step + 2 steps
3. 2 steps + 1 step



Leetcode 70 - Climbing Stairs (top-down)

```
class Solution {
private:
    int climb(int n, vector<int> &memo) {
        if (memo[n] != -1) return memo[n];
        memo[n] = climb(n - 2, memo) + climb(n - 1, memo);
        return memo[n];
    }

public:
    int climbStairs(int n) {
        vector<int> memo(n + 1, -1);
        memo[0] = 1;
        memo[1] = 1;

        return climb(n, memo);
    }
};
```



Leetcode 70 - Climbing Stairs (bottom-up)

```
class Solution {
public:
    int climbStairs(int n) {
        vector<int> dp(n + 1);
        dp[0] = 1;
        dp[1] = 1;

        for (int i = 2; i <= n; i++) {
            dp[i] = dp[i - 1] + dp[i - 2];
        }

        return dp[n];
    }
};
```



Leetcode 70 - Climbing Stairs (bottom-up; OP)

```
class Solution {
public:
    int climbStairs(int n) {
        int prev = 1, curr = 1;

        for (int i = 2; i <= n; i++) {
            int tmp = curr + prev;
            prev = curr;
            curr = tmp;
        }

        return curr;
    }
};
```



Leetcode 746 - Min Cost Climbing Stairs

與上一題不同，這一題的每一階都有不同的價格。有一個 n 階的樓梯，從第 i 階可以花費 $\text{cost}[i]$ 元踩一階或跨兩階。請問到頂的最低花費為何？（即找出最優路徑之花費）

Example 1:

Input: $\text{cost} = [10, 15, 20]$
Output: 15
Explanation: You will start at index 1.
- Pay 15 and climb two steps to reach the top.
The total cost is 15.

Example 2:

Input: $\text{cost} = [1, 100, 1, 1, 1, 100, 1, 1, 100, 1]$
Output: 6
Explanation: You will start at index 0.
- Pay 1 and climb two steps to reach index 2.
- Pay 1 and climb two steps to reach index 4.
- Pay 1 and climb two steps to reach index 6.
- Pay 1 and climb one step to reach index 7.
- Pay 1 and climb two steps to reach index 9.
- Pay 1 and climb one step to reach the top.
The total cost is 6.



Leetcode 746 - Min Cost Climbing Stairs (top-down)

```
class Solution {
private:
    int minCost(vector<int> &cost, vector<int> &memo, int step) {
        if (step == 0 || step == 1) {
            return cost[step];
        }
        if (memo[step] != -1) {
            return memo[step];
        }
        memo[step] = min(minCost(cost, memo, step - 1), minCost(cost, memo, step - 2)) + cost[step];
        return memo[step];
    }
public:
    int minCostClimbingStairs(vector<int>& cost) {
        vector<int> memo(cost.size(), -1);
        return min(minCost(cost, memo, cost.size() - 1), minCost(cost, memo, cost.size() - 2));
    }
};
```



Leetcode 746 - Min Cost Climbing Stairs (bottom-up)

```
class Solution {
public:
    int minCostClimbingStairs(vector<int>& cost) {
        vector<int> dp(cost.size());
        dp[0] = cost[0];
        dp[1] = cost[1];

        for (int i = 2; i < cost.size(); i++) {
            dp[i] = min(dp[i - 1], dp[i - 2]) + cost[i];
        }

        return min(dp[cost.size() - 1], dp[cost.size() - 2]);
    }
};
```